CSC3110 Final — Breakdowns · 13 algorithms

DP vs D&C: both split — D&C subproblems **independent**; DP because they **overlap**. · **Greedy vs DP:** greedy optimizes the **next step** (needs greedy-choice property); DP the **whole path** via a table.

1. Topological Sort · Decrease&C

socks before shoes

Linear order of a **DAG**; every edge u→v has u before v.
Source-removal: ① find in-degree-0 vertex ② output & remove it (drop out-edges) ③ new sources appear ④ repeat. No source + vertices left ⇒ **cycle**.
Θ(V+E)

2. Hoare Partition · Divide&C

reference person

Quicksort engine: `p=A[l]`, ≤p left / ≥p right; pivot lands final.
`i=l, j=r+1`. i→ to first A[i]≥p; j← to first A[j]≤p. i<j swap & repeat. i≥j → swap A[l],A[j] = **split**.
Θ(n log n) avg·Θ(n²) worst

3. Bottom-up Heapify · Transform&C

corporate reorg

Max-heap array: parent≥children, children of i at 2i+1,2i+2.
i=⌊n/2⌋−1 down to 0: **sift down** vs **larger** child, swap if smaller, continue; stop at leaf or ≥ both.
build Θ(n)·sort Θ(n log n)

4. Kruskal MST · Greedy

cheapest roads, skip loops

Min-weight acyclic connected subgraph, |V|−1 edges.
① sort edges ↑weight ② add if **no cycle** (union-find) ③ stop at |V|−1 edges.
Θ(E log E)

5. Prim MST · Greedy

spreading vine

Grow **one** tree; add cheapest edge tree→outside.
① start any vertex ② min-weight edge to an outside vertex ③ add edge+vertex ④ until |V|−1. Prim=one tree; Kruskal=fragments.
Θ(E log V)

6. Comparison Counting Sort · Space-Time

how many did you beat?

Count elements smaller than each → that count is its index.
Pairs i<j: A[i]<A[j]⇒Count[j]++ else Count[i]++. Place A[i] at Count[i].
Θ(n²) · distrib. Θ(n+range)

7. Coin-Row DP · Dynamic Prog.

buffet, no adjacent

Max value from a row, no two adjacent.
$F(i)=\max(F(i-1),\ c_i+F(i-2))$, $F(0)=0, F(1)=c_1$. Recover: F(i)≠F(i−1)⇒took i, jump −2.
Θ(n)

8. Change-Making DP · Dynamic Prog.

fewest jug pours

Fewest coins for amount m (unlimited).
$F(m)=\min_{c_j\le m} F(m-c_j)+1$, F(0)=0. +1=last coin; $m-c_j$=amount before it. Greedy fails on {1,3,4}.
Θ(n·amount)

9. Assignment · B&B · Backtracking

wedding seating + shortcut

Assign n people→n jobs, min cost; beat n! by pruning.
Bound=committed + each unassigned person's cheapest **free** job (may double-use cols). Branch person-by-person; expand smallest bound; **prune** if ≥ incumbent.
exp. worst, prunes in practice

10. Dijkstra · Greedy

pond ripples

Shortest path source→all (no neg weights).
① dist(s)=0 rest ∞, keep prev[] ② pick smallest-dist unvisited (final) ③ relax dist(u)+w<dist(v) ④ repeat.
Θ(E log V) · no neg edges

11. Huffman Coding · Greedy

merge two smallest

Optimal prefix code: frequent short, rare long.
① min-heap of freqs ② extract 2 smallest, parent=sum (left0/right1), reinsert ③ until 1 root. Code=root→leaf. Bits=Σ freq×len.
Θ(n log n)

12. 0/1 Knapsack DP · Dynamic Prog.

pack suitcase

Max value subset within capacity W.
$F[i][c]=\max(F[i-1][c],\ v_i+F[i-1][c-w_i])$ if $w_i\le c$.
Row0/col0=0. Recover: F[i][c]≠F[i−1][c]⇒took i, drop w_i.
Θ(nW) pseudo-poly

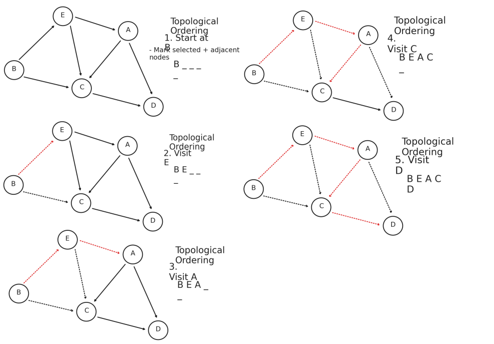
13. Floyd & Warshall · Dynamic Prog.

open hubs 1 at a time

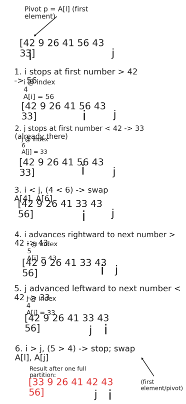
All-pairs via intermediates 1..k (k outer loop).
Floyd: `D[i][j]=min(D[i][j],D[i][k]+D[k][j])`.
Warshall: R[i][j]=R[i][j] OR (R[i][k] AND R[k][j]).
Θ(n³) time·Θ(n²) space

CSC3110 Final — Visuals · worked-example diagrams

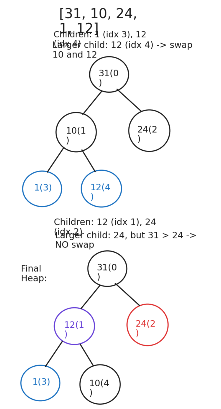
1. Topological Sort



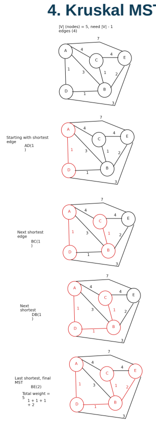
2. Hoare Partition



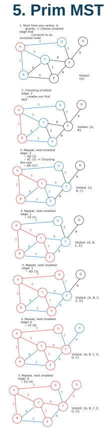
3. Bottom-up Heapify



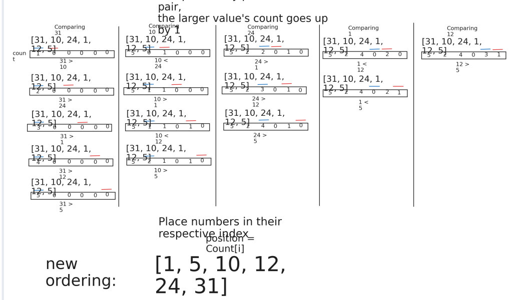
4. Kruskal MST



5. Prim MST



6. Comparison Counting Sort



7. Coin-Row DP

{3, 2, 5, 2, 1, 2, 4}

Coin-row problem — dynamic programming

F(i) = max(F(i−1) , c_i + F(i−2))   F(0)=0, F(1)=c_1

Rule: pick the maximum total value, but no two adjacent coins.

Coin row — position (above) / value (in box). Green = selected coins.

DP table — fill F(i) left to right:

i	0	1	2	3	4	5	6	7
c_i	–	3	2	5	2	1	2	4
F(i)	0	3	3	8	8	9	10	13

backtrace: jump back 2 each time a coin is taken

Forward computation (which branch wins each max):

```
F(1) = c₁              = 3
F(2) = max( 3 , 2+0 )  = 3
F(3) = max( 3 , 5+3 )  = 8
F(4) = max( 8 , 2+3 )  = 8
F(5) = max( 8 , 1+8 )  = 9
F(6) = max( 9 , 2+8 )  = 10
F(7) = max( 10 , 4+9 ) = 13
```

Which coins? (backtrace)

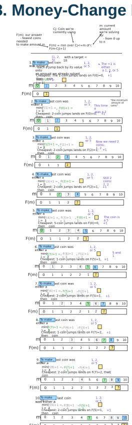
start at F(7); if the “take” branch won, grab that coin and jump back two positions.

F(7)→take c_7 → F(5)→take c_5 → F(3)→take c_3 → F(1)→take c_1

⚠ note: coin 6 wins its forward max but is NOT selected — the backtrace jumps 7→5, skipping it.

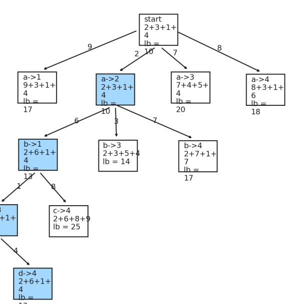
✓ Max value = 13 — coins at positions 1, 3, 5, 7 = 3 + 5 + 1 + 4

8. Money-Change DP

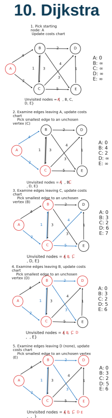


9. Assignment B&B

	Job 1	Job 2	Job 3	Job 4
Person a	9	2	7	8
Person b	6	4	3	7
Person c	5	8	1	8
Person d	7	6	9	4



10. Dijkstra



11. Huffman

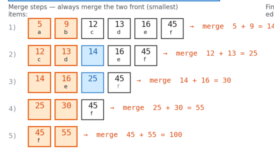
Huffman Coding — step by step (replicable recipe)

The recipe (works for ANY frequency set):

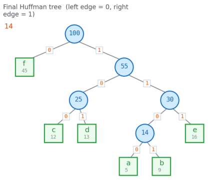
```
1.  List every symbol with its frequency.
2.  Put all symbols in a queue, sorted smallest-first.
3.  Remove the TWO smallest; make a parent = their sum.
       (first removed = left/0 child,  second = right/1 child)
4.  Insert the parent back into the sorted queue.
5.  Repeat 3–4 until ONE node remains = the root.
6.  Code of a symbol = edges root→leaf (left=0, right=1).
7.  Total bits = Σ (frequency × code length).
```

Frequencies: {a:5, b:9, c:12, d:13, e:16, f:45}

Merge steps — always merge the two front (smallest) items:



Final Huffman tree (left edge = 0, right edge = 1)



Read each code root→leaf, then total the bits:

symbol	freq	code	length	freq×len
f	45	0	1	45
c	12	100	3	36
d	13	101	3	39
e	16	111	3	48
a	5	1100	4	20
b	9	1101	4	36

Total = 45+36+39+48+20+36 = 224 bits (fixed 3-bit code would be 300)

12. 0/1 Knapsack

0/1 Knapsack — dynamic programming (replicable recipe)

The recipe (each item is taken whole or left — no fractions):

```
1.  Rows = items 1..n, columns = capacity 0..W.
2.  Row 0 and column 0 are all 0  (no items, or no space).
3.  For each cell F[i][c] = max of:
        skip item i :  F[i−1][c]
        take item i :  value[i] + F[i−1][c − weight[i]]   (only if weight[i] ≤ c)
4.  Answer = F[n][W]  (bottom-right cell).
5.  Backtrace: if F[i][c] ≠ F[i−1][c], item i was TAKEN → drop its weight;
        otherwise it was skipped → move straight up a row.
```

Items (W = 7)

item	weight	value
1	1	1
2	2	6
3	3	10
4	5	16

green = chosen items

DP table — fill row by row, left to right:

item \ capacity	0	1	2	3	4	5	6	7
i=0 (none)	0	0	0	0	0	0	0	0
i=1 (w1,v1)	0	1	1	1	1	1	1	1
i=2 (w2,v6)	0	1	6	7	7	7	7	7
i=3 (w3,v10)	0	1	6	10	11	16	17	17
i=4 (w5,v16)	0	1	6	10	11	16	17	22

orange path = backtrace

Example cell — the choice at F[4][7]:

F[4][7] = max(skip: F[3][7]=17 , take: 16 + F[3][2]=16+6=22) = 22 → item 4 taken

Backtrace from F[4][7]:

22≠17 → take item4 (c:7→2)   6=6 → skip item3   6≠1 → take item2 (c:2→0)   done

✓ Max value = 22 — items {2, 4} (weights 2+5 = 7 ≤ 7, values 6+16 = 22)